Scalability

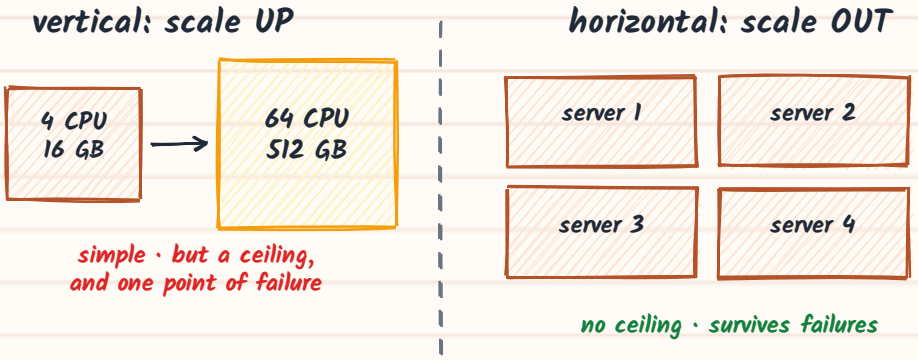
Lesson 3 · vertical vs horizontal, stateless servers & the scale cube

In this lesson → ① SnapLink grows ② Up vs out ③ Comparison ④ Stateless servers ⑤ Sticky sessions ⑥ Next bottleneck ⑦ Scale cube ⑧ Latency vs throughput ⑨ Autoscaling
Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

1 What is scalability?

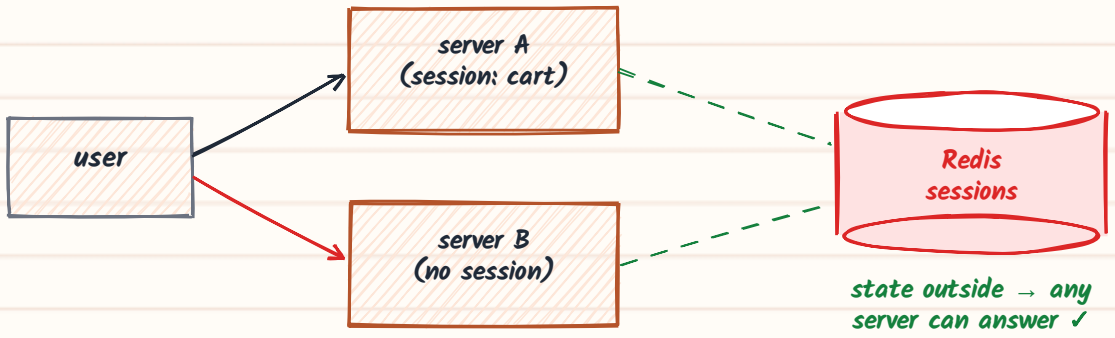
> **Scalability** = the ability to handle more load by adding resources, without redesigning the system.

2 Vertical vs horizontal



	Vertical (scale up)	Horizontal (scale out)
How	bigger machine	more machines + load balancer
Limit	hardware ceiling, cost grows fast	nearly unlimited
Failure	single point of failure	others keep serving
Complexity	low, no code change	needs stateless app, LB
Typical	databases (first)	web / app servers

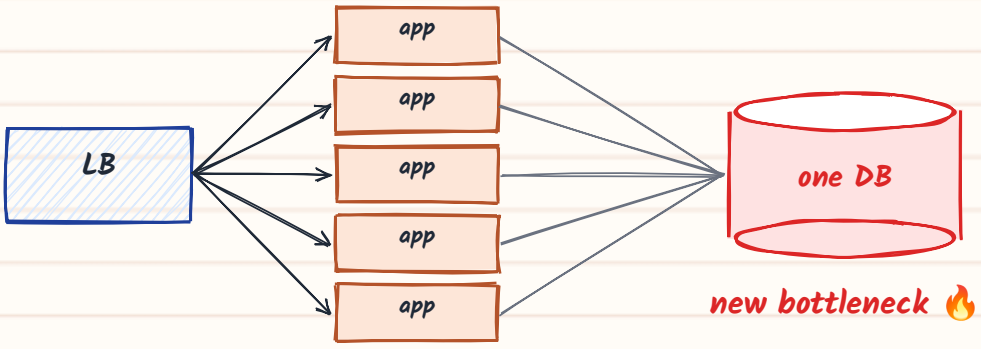
3 Stateless servers



- ✓ No user/session data in server memory
- ✓ Sessions in Redis / DB, or signed tokens (JWT)
- ✓ Files in object storage, not local disk
- ✓ Any server can answer any request → easy scale out/in

Sticky sessions?
LB pins a user to one server. Works, but uneven load and sessions lost if the server dies. Prefer stateless + shared store.

4 Scaling moves the bottleneck



→ App tier scaled → DB becomes the bottleneck

→ Next fixes: cache (SD-15), read replicas (SD-12), sharding (SD-13)

→ Scaling = find bottleneck → fix → repeat

5 The scale cube

X: clone

same app, many copies

load balancer

Y: split by function

users · photos · feed services

microservices

Z: split by data

users A–M / N–Z

sharding

Axis	Idea	Example
X	clone the app N times	app servers behind LB
Y	split by function	user, photo, feed services
Z	split by data	shard users by ID range / hash

6 Latency vs throughput

	Latency	Throughput
Means	time for one request	requests per second
Unit	ms (p50, p99)	QPS / RPS
Improve by	cache, closer data, less work	more servers, parallelism

7 Autoscaling

- ✓ Scale on CPU, QPS or queue length
- ✓ Keep a minimum; set a maximum (cost)
- ✓ Pre-scale for known peaks (sales, events)
- ✓ Scale in slowly to avoid flapping

8 Interview Corner

Q. Vertical vs horizontal scaling?

A. Bigger machine vs more machines; horizontal has no hard ceiling and tolerates failures.

Q. Why stateless services?

A. Any instance can serve any request, so instances can be added/removed freely.

Q. Where does state go?

A. Shared stores: Redis/DB for sessions, object storage for files, or client tokens.

Q. What happens after scaling the app tier?

A. The database usually becomes the next bottleneck.

Q. Latency vs throughput?

A. Time per request vs requests per second; adding servers mainly raises throughput.

⚡ Revise in 60 seconds

- → **Up**: bigger box, ceiling
- → **Out**: more boxes, resilient
- → **Stateless** app servers
- → State → Redis / DB
- → Bottleneck moves → DB
- → Scale cube **X · Y · Z**
- → Latency ≠ throughput
- → Autoscale + minimum

Next lesson → SD-04
Availability & reliability: SLAs, nines, redundancy.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**